

Scalable Parallel-in-Time Integration for Equations of Motion

Nathaniel W. Chapman

Department of Computer Science
Central Washington University

2025-08-11

Table of Contents

Introduction

Background

The Parareal Algorithm

The Parareal Algorithm at Scale

Performance Analysis

Conclusion

Content

Introduction

Background

The Parareal Algorithm

The Parareal Algorithm at Scale

Performance Analysis

Conclusion

It's About Time

The main issue

Calculations take time...and we only have so much.

Minimizing Time

- Total runtime = $\frac{\text{time}}{\text{calculation}} * \text{number of calculations}$
- Want to minimize runtime
- Minimizing time / calculation $\iff$
Speeding up sequential evaluation
- Maximizing calculations / time $\iff$
Adding more parallelization

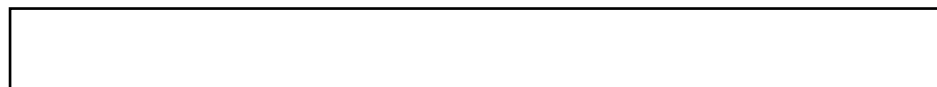


Figure 1:

Parallelizing Time

- Physics depends on time and space
- Space has been parallelized
- Time has remained sequential
- Parallel-in-Time integration offers a new avenue to speedup



Figure 2:

Maximizing Resource Utilization

- Space discretized as much as it can
e.g. constrained by CFL
- Spatial parallelization saturated
- Remaining cores sit idle

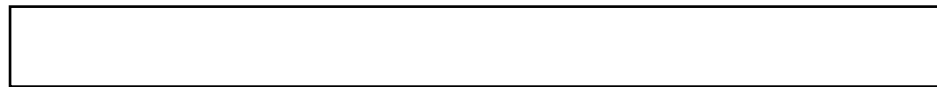


Figure 3:

More Threads is Always the Answer

- PinT scales with threads
- CPUs have ~10 threads
- GPUs have ~10,000 threads
- Distributed has ∞ threads



Figure 4:

This work

- Implement the Parareal Algorithm
- Use GPUs and distributed systems
- Simulate motion

Content

Introduction

Background

The Parareal Algorithm

The Parareal Algorithm at Scale

Performance Analysis

Conclusion

Growth of Parallel-in-Time Integration

- PinT is growing
- Not a new idea
- More cores $\Rightarrow$ more interest
- On track for more growth

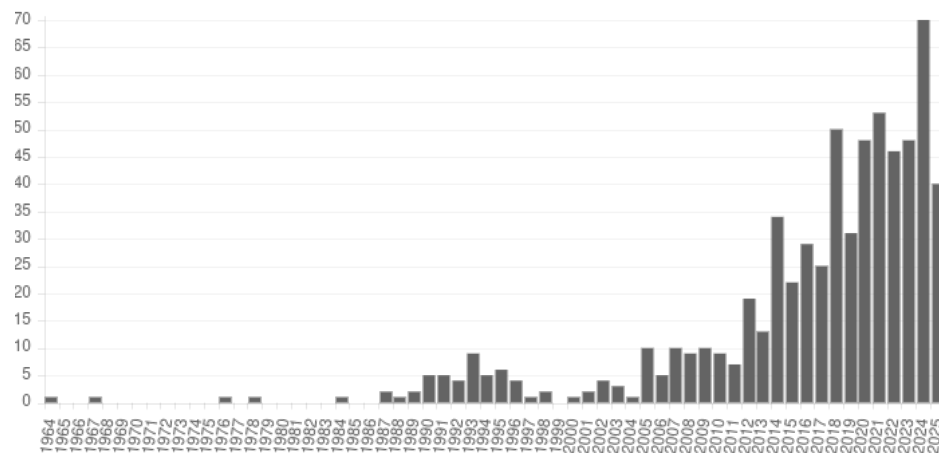


Image credit: parallel-in-time.org

Applications of Parallel-in-Time Integration

- Applications in many sciences
- **Physics:** E&M, Plasmas, Fluids, Materials, Astrophysics
- **Engineering:** Manufacturing
- **Operations:** Optimal control
- **Economics:** Game theory, Market dynamics
- **Biology:** Animal patterns, Blood flow in fish
- **Machine Learning:** Training neural networks

Methods of Parallel-in-Time Integration

- Specialized methods
- **General:** Parareal, PITA, PFASST, RIDC
- **Hyperbolic:** ParaDiag, MGRIT
- **Parabolic:** STMG, WRMG

Diagonalization-based Parallel-in-Time Methods ParaDiag: diagonalization-based Parallel-in-Time (PinT) algorithms, which can handle both both dissipative and hyperbolic equations	Multigrid Reduction in Time (MGRIT) Multigrid Reduction in Time algorithm developed at the LLNL	PFASST PFASST is a time-parallel expansion of spectral deferred corrections methods.
PITA The parallel implicit time-integrator.	Parareal Parareal is one of the most widely studied time-parallel methods.	Revisionsit Integral Deferred Correction (RIDC) RIDC
Space-time Multigrid (STMG) STMG is a time-parallel method for parabolic PDEs.	Space-time concurrent multigrid waveform relaxation (WRMG) WRMG is normally only a space-concurrent method; time parallelism is possible using cyclic reduction.	Want your method to show up here? You can do it yourself!

Image credit: parallel-in-time.org

Implementations of Parallel-in-Time Integration

- Different languages with different means of parallelism
- **Languages:** Fortran, C, C++, Python
- **Parallelism:** OpenMP, MPI
- **HPC:** Very few

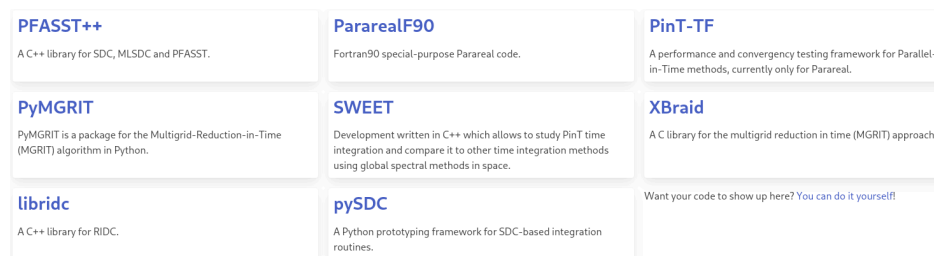


Image credit: parallel-in-time.org

Language of Parallel-in-Time Integration

- Traditional HPC languages have been used e.g. C/C++, Fortran
- Python has been used
- Julia has the speed of C with the ease of Python
- Julia natively supports parallel, GPU, and distributed
- PinT should be implemented in Julia

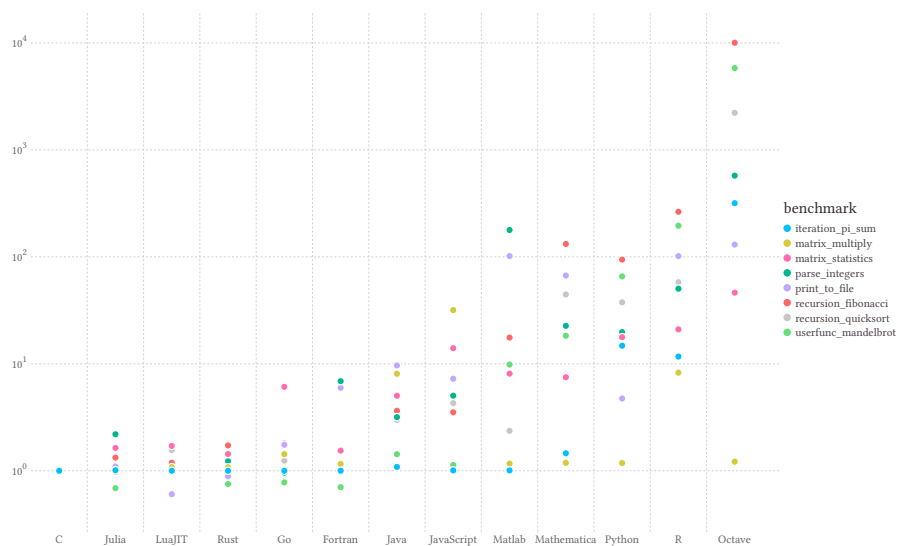


Image credit: julialang.org

This Contribution to Parallel-in-Time Integration

- **Application:** Physics of motion
- **Method:** Parareal
- **Parallelism:** Distributed GPUs
- **Language:** Julia

Content

Introduction

Background

The Parareal Algorithm

The Parareal Algorithm at Scale

Performance Analysis

Conclusion

Equations of Motion

- What is an equation of motion?
- **Physics:** A constraint on the dynamics of an object interacting with its environment
- **Math:** A second-order, hyperbolic, differential equation
- Solution is completely determined by initial values (for classical physics)

Ex. The wave equation

$$\frac{\partial^2}{\partial t^2} \phi - \frac{1}{c^2} \nabla^2 \phi = 0$$

Ex. Newton's Second Law

$$\sum \vec{F} = m\vec{a}$$

Traditional Numerical Integration

- Future space only depends on previous space
- Space can be parallelized because we know its previous value
- Time has to be sequential because we don't

The discretized 1D wave equation

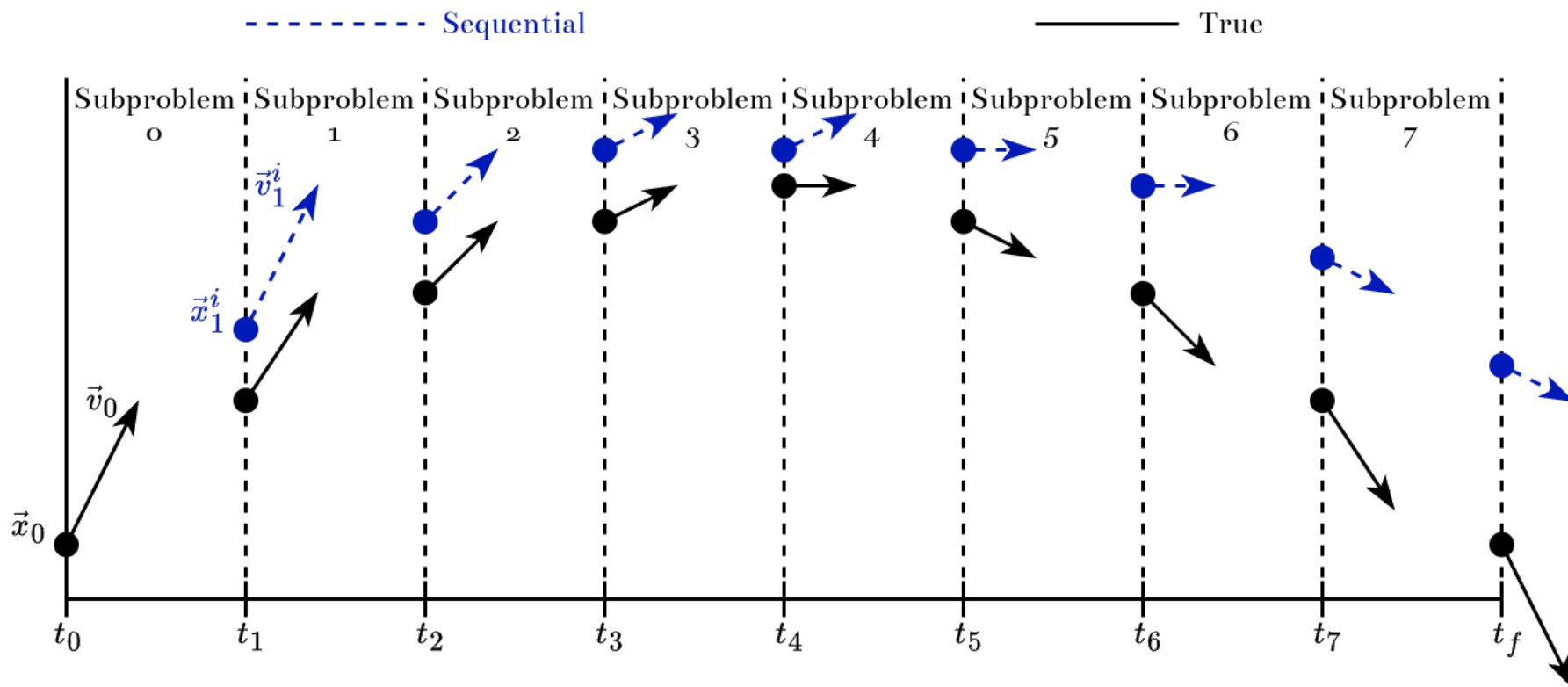
$$\frac{\phi_{x_j}^{t_{i+1}} - 2\phi_{x_j}^{t_i} + \phi_{x_j}^{t_{i-1}}}{\Delta t^2} = \frac{1}{c^2} \frac{\phi_{x_{j+1}}^{t_i} - 2\phi_{x_j}^{t_i} + \phi_{x_{j-1}}^{t_i}}{\Delta x^2}$$
$$\Rightarrow \phi_j^{i+1} = 2\phi_j^i - \phi_j^{i-1} + \left(\frac{\Delta t}{c\Delta x}\right)^2 (\phi_{j+1}^i - 2\phi_j^i + \phi_{j-1}^i)$$

The Parareal Algorithm

- What about approximating the future?
- The Parareal Algorithm
 1. Make inaccurate prediction sequentially
 2. Make accurate predictions in parallel based on those values
 3. Correct inaccurate prediction with accurate ones
 4. Loop until converged

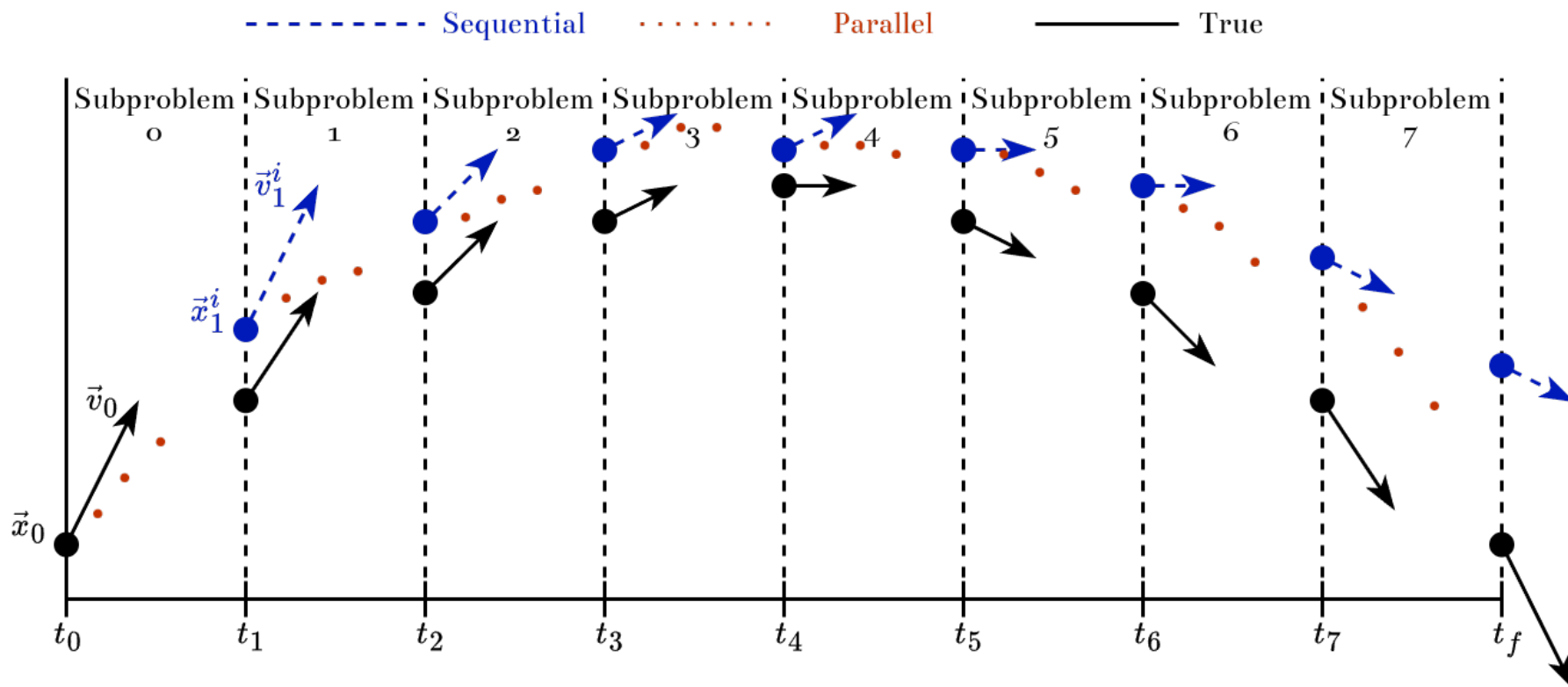
The Parareal Algorithm - Prepare the Subproblems

- Use fast, inaccurate, sequential “coarse” solver to approximate solution



The Parareal Algorithm - Solve the Subproblems

- Use slow, accurate, sequential “fine” solver on each coarse value simultaneously



The Parareal Algorithm - Correct the Coarse Solution

- Use cheap, inaccurate “coarse” solver but correct each propagation
- Correction term = difference between previous iteration’s fine and coarse values
- “Fine deviation decreases each iteration”

Literature

$$u_t^i := \underbrace{\mathcal{G}(u_{t-1}^i)}_{\text{predictor}} + \underbrace{\mathcal{F}(u_{t-1}^{i-1}) - \mathcal{G}(u_{t-1}^{i-1})}_{\text{corrector}}$$

$$u_t^i := \underbrace{\mathcal{F}(u_{t-1}^{i-1})}_{\text{fine solution}} + \underbrace{\mathcal{G}(u_{t-1}^i) - \mathcal{G}(u_{t-1}^{i-1})}_{\text{deviation}}$$

Alternatively

The Parareal Algorithm - Converge the Coarse Solution

- Keep iterating until coarse solution doesn't change much
- Max iterations = coarse discretization
- Converges to “direct” fine solution

Convergence condition

$$\max_{1 \leq t \leq N-1} |u_t^i - u_t^{i-1}| < \varepsilon$$

Content

Introduction

Background

The Parareal Algorithm

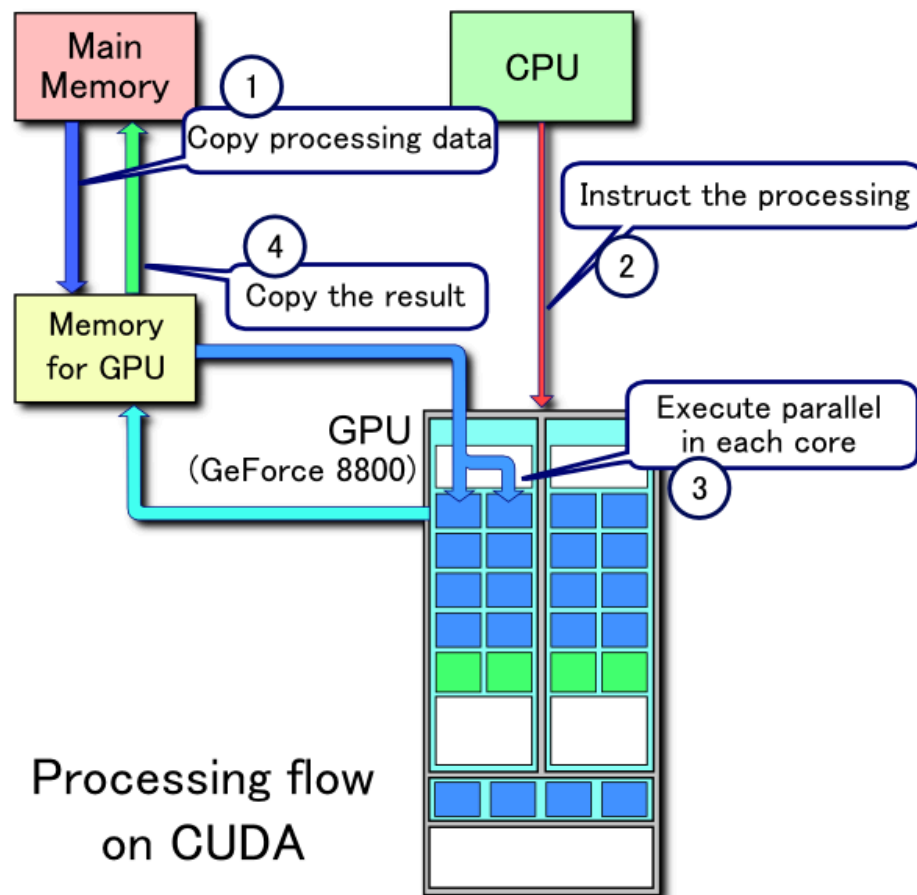
The Parareal Algorithm at Scale

Performance Analysis

Conclusion

GPU Computing

- Execute the same “kernel” program simultaneously on different data
- “Add 1 to every element of an array” executes in parallel
- Data copies between GPU and host system
 - Takes a lot of time

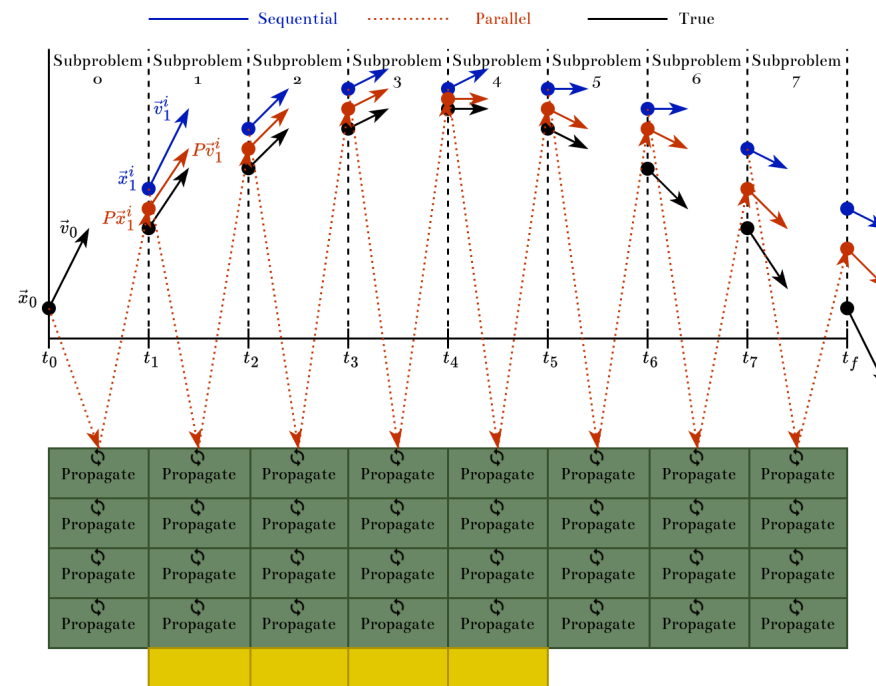


Processing flow
on CUDA

Image Credit: Wikipedia *CUDA*

Parareal on the GPU

- Almost identical to running on CPU
- Fine solve $\rightarrow$ Copy, fine solve, copy
- **Pro:** Can be orders-of-magnitude more accurate in same amount of time!
- **Con:** Copying data takes significant time each iteration!
- **Fix:** Copy enough and do enough to make it worth it!



Distributed Computing

- Shared memory to unshared memory
- Need to explicitly copy data between processes
- Remote Procedure Calls
 - Heuristic: “Evaluate source code over there”
 - Watch out for references!
- Communicating over network is really slow!

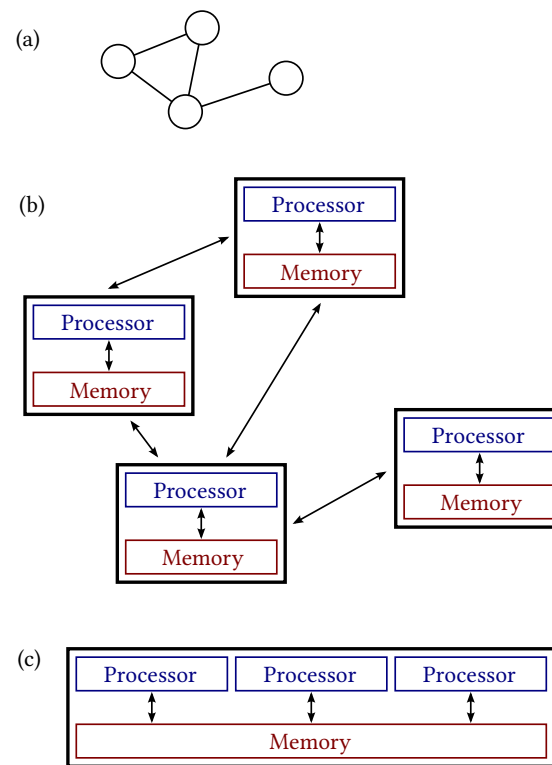


Image Credit: Wikipedia *Distributed Computing*

Parareal on the Cluster

- Almost identical to running on GPU
 1. Director executes coarse solver
 2. Director distributes subproblems to workers
 3. Director tells workers to execute fine solver on their GPUs
 4. Director requests results
 5. Director corrects and loops

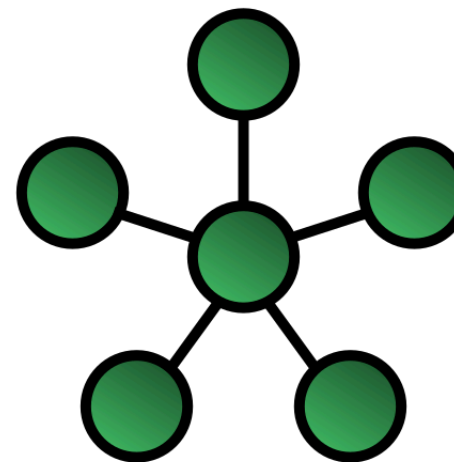


Image Credit: Wikipedia *Star Network*

Content

Introduction

Background

The Parareal Algorithm

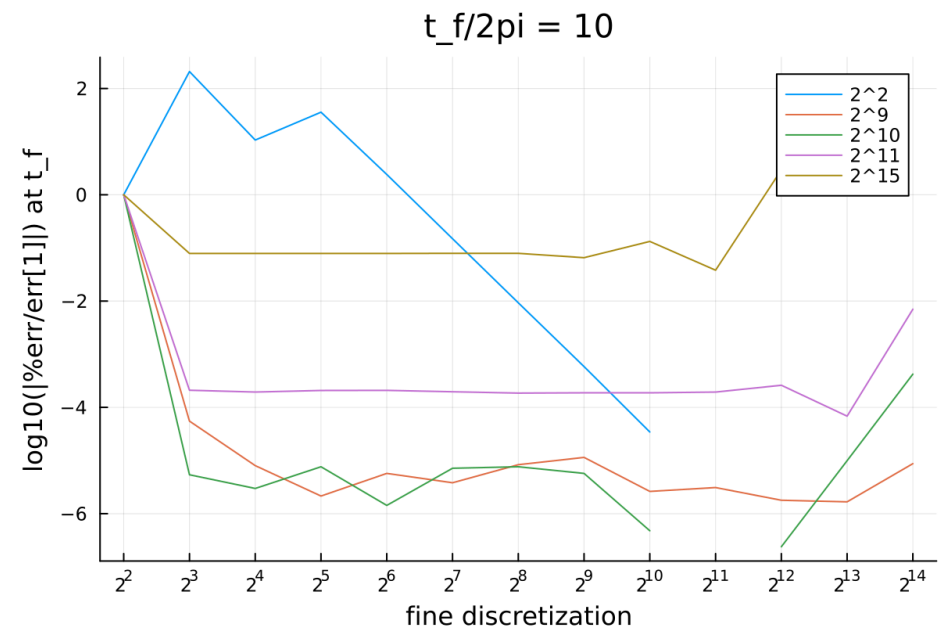
The Parareal Algorithm at Scale

Performance Analysis

Conclusion

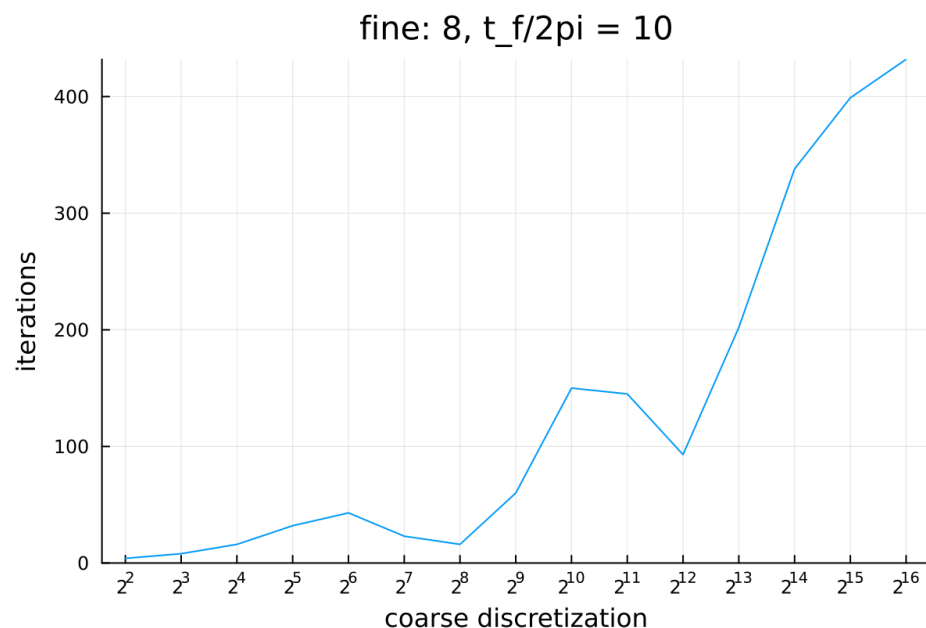
Numerical Analysis - Error & Energy Drift

- Simple pendulum over 10 oscillations
- Energy is conserved $\Rightarrow$ deviations are from numerical error
- Error decreases by several orders-of-magnitude even for small fine discretizations for almost all coarse discretizations
- Decreases decrease as coarse discretization increases



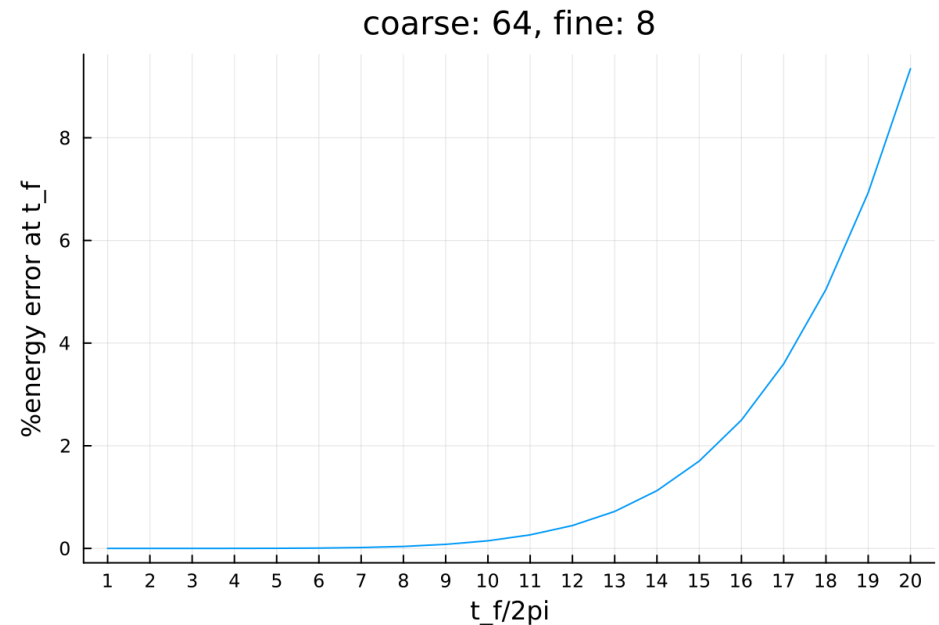
Numerical Analysis - Convergence

- Linear convergence for small coarse discretizations
i.e. iterations $\approx$ discretization
- Superlinear convergence
(i.e. iterations $\ll$ discretization) for larger coarse discretization
- Results consistent with literature
- Iterations blows up for high coarse discretization due to rounding error from using 32-bit floats



Numerical Analysis - Stability

- Constant time step, but final time increases
- No change in local error
- Significant change in global error i.e. error at the final time
- Relative error increases exponentially with final time



Latency & Data Transfers - Host-Device Transfers

- Data is transferred between the host and the device every iteration.
- Pinned and Zero-copy memory are methods that could mitigate this latency
- Pinned memory prevents data from being moved out of RAM
- Zero-copy memory allows both CPU and GPU to refer to the same data

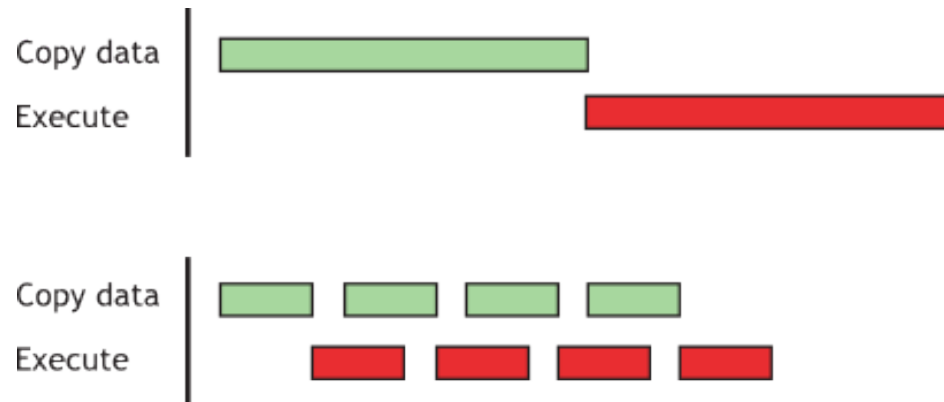
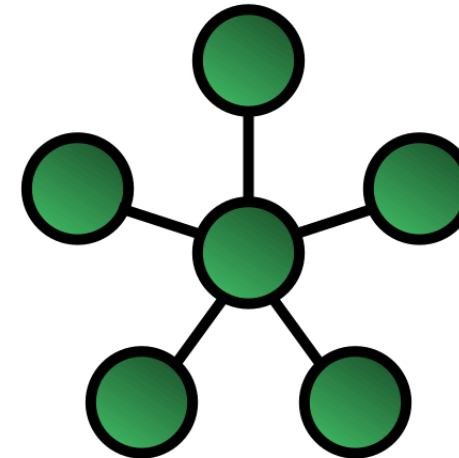


Figure 18: Image Credit: CUDA Documentation

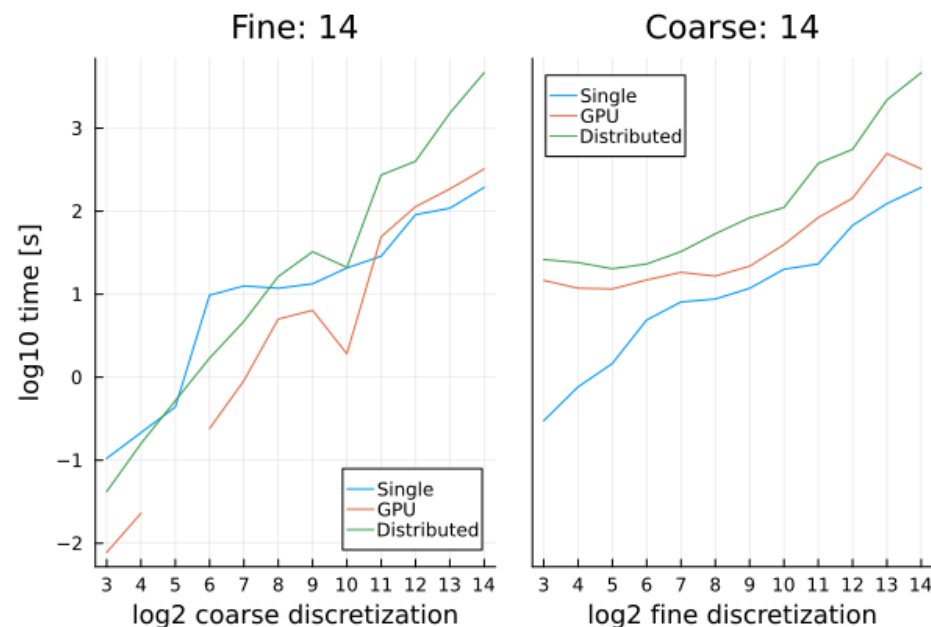
Latency & Data Transfers - Host-Host Transfers

- Cluster topology can have a significant impact on performance
- Distinction between the logical topology and the physical topology
- Data actually routes through the physical topology regardless of how workers are connected logically
- Longer physical paths $\Rightarrow$ more time transferring



Benchmarks - Comparison

- Comparing the different approaches to parallelism
- Surprisingly single threaded has the least runtime
- Significant overhead in using these parallelization methods
- Slow convergence due to round-off error at high discretizations does not help.



Content

Introduction

Background

The Parareal Algorithm

The Parareal Algorithm at Scale

Performance Analysis

Conclusion

Conclusion

- The field of PinT is growing
- PinT methods have found their use in many fields of science
- Bigger problems need time parallelization to maximize resource utilization

Future Work

- Use traditional languages such as C, C++, Fortran to increase raw performance
- Use vendor agnostic GPU implementations

Acknowledgements

- Dr. Andy Piacsek
- The CWU CS Department
- The CWU Physics Department

Thank You

- Thank you for your time
- This work and its code `PararealGPU.jl` are available on GitHub (→) and at pararealgpu.computationalphysics.net

